

Tugas Besar 2

nama_kelompok

IF2211 Strategi Algoritma

DOMTracer : BFS-DFS Based CSS Selector Engine

Semester 2 2025/2026



Disusun oleh:

nama_kelompok

Moh. Hafizh Irham Perdana	(13524025)
Aufa Rienaldifaza Ahmad	(13524027)
Muhammad Aufar Rizqi Kusuma	(13524061)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



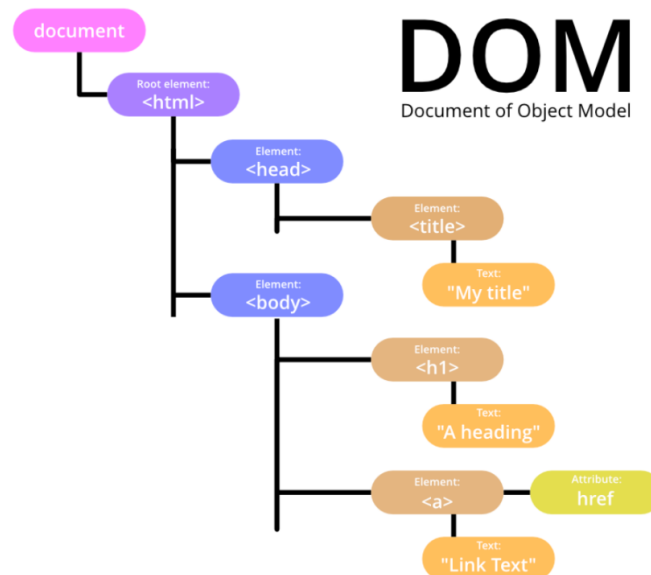
Daftar Isi

1	Pendahuluan	2
1.1	Deskripsi Tugas	2
1.2	Spesifikasi Wajib	5
1.3	Spesifikasi Bonus	6
2	Dasar Teori	8
3	Aplikasi BFS dan DFS	9
4	Implementasi dan Pengujian	10
5	Kesimpulan dan Saran	11
5.1	Kesimpulan	11
5.2	Saran	12
6	Lampiran	13
6.1	Tabel Checklist	13
6.2	Tautan GitHub	13

Bab 1

Pendahuluan

1.1 Deskripsi Tugas



Gambar 1.1: Struktur Pohon DOM

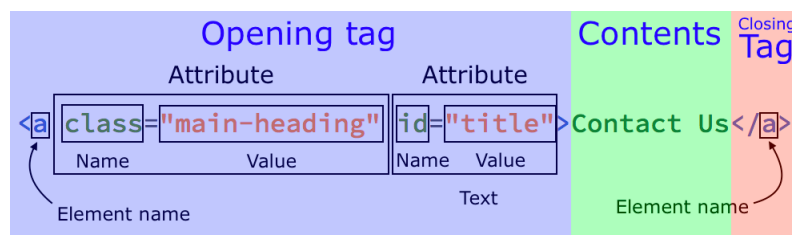
Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram. Struktur sebuah file HTML pada umumnya meliputi elemen root `<html>` yang berisi dua bagian utama yaitu `<head>` dan `<body>`. Bagian `<head>` berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan `<body>` berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen

pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector. Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. **Tags** pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

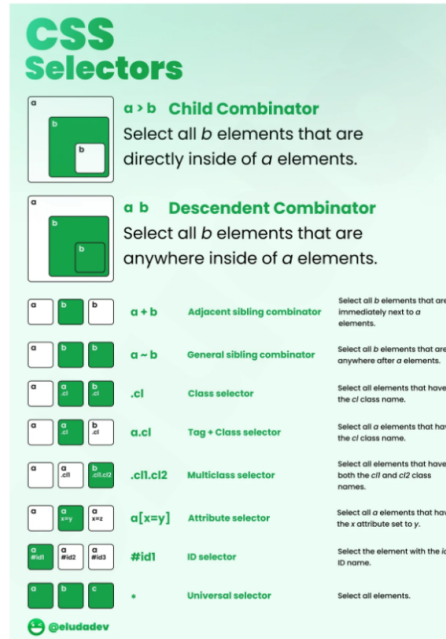


Gambar 1.2: Struktur Pohon DOM

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (<>). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring/sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling. Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. **CSS Selector** adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak.



Gambar 1.3: Struktur Pohon DOM

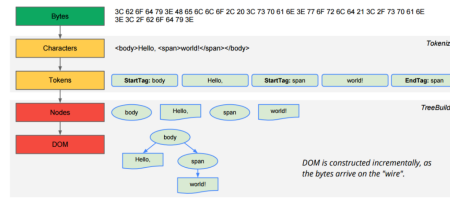
Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector.** Memilih elemen berdasarkan nama tag HTML. Contoh: `p` memilih semua elemen `<p>`.
- Class Selector.** Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (`.`). Contoh: `.box`.
- ID Selector.** Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (`#`). Contoh: `#header`
- Universal Selector.** Memilih semua elemen HTML. Contoh: `*`.

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari list berikut ini.

- Child Combinator (`>`)
- Descendent Combinator (`" "`)
- Adjacent Sibling Combinator (`+`)
- General Sibling Combinator (`~`)

- HTML Parsing** adalah Proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.



Gambar 1.4: Struktur Pohon DOM

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen `<html>`. Dari node root tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

1.2 Spesifikasi Wajib

Pada tugas ini, mahasiswa diminta untuk membuat aplikasi traversal pohon HTML (DOM Tree) yang menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS) untuk melakukan pencarian elemen berdasarkan CSS selector pada struktur DOM.

- Buatlah sebuah aplikasi berbasis web yang terdiri dari frontend bebas dan backend dalam bahasa Go, C#, Rust, atau Typescript yang mengimplementasikan algoritma BFS dan DFS untuk melakukan penelusuran CSS pada pohon DOM.
- Aplikasi dapat melakukan scraping HTML dari sebuah website melalui URL yang diberikan oleh pengguna.
- Secara umum, berikut adalah input yang diharapkan.
 - URL website atau memasuki teks HTML sendiri
 - Pilihan algoritma traversal (BFS / DFS)
 - CSS selector
 - Jumlah hasil yang diinginkan:
 - * Top n kemunculan
 - * Semua kemunculan

- HTML yang diperoleh kemudian diparsing menjadi struktur data pohon (DOM Tree).
- Output yang diharapkan adalah sebagai berikut.
 1. Aplikasi dapat menampilkan visualisasi struktur pohon DOM, disertai dengan keterangan kedalaman maksimum tree.
 2. Aplikasi dapat menampilkan highlight pada jalur yang di-traversal oleh algoritma penelusuran yang dipilih untuk menandakan elemen-elemen yang terpengaruh.
 3. Aplikasi dapat menampilkan waktu pencarian serta banyak node yang dikunjungi.
 4. Setelah melakukan penelusuran, aplikasi menyimpan sebuah traversal log yang memiliki informasi tahapan penelusuran yang dilakukan.
- Repository bagian frontend dan backend boleh digabung/dipisah.
- Setiap kelompok harap mengisi nama kelompok dan anggotanya pada link berikut, dengan maksimal tanggal Minggu, 5 April 2026 23.59. Nama kelompok dilarang mengandung unsur SARA, penghinaan, provokasi, atau hal-hal lain yang bertentangan dengan norma hukum.
- Perlu diperhatikan bahwa kelompok tubes untuk mata kuliah ini tidak boleh sama. Jadi jika berkelompok dengan X pada tubes ini, maka pada tubes selanjutnya anda tidak diperkenankan berkelompok dengan X.

1.3 Spesifikasi Bonus

- **(4 poin) [Video]** Membuat video tentang aplikasi BFS dan DFS pada penelusuran pohon DOM kemudian mengunggahnya di Youtube. Video dibuat harus memiliki audio dan menampilkan wajah dari setiap anggota kelompok. Untuk contoh video tubes stima tahun-tahun sebelumnya dapat dilihat di Youtube dengan kata kunci “Tubes Stima”, “strategi algoritma”, “Tugas besar stima”, dll. Semakin menarik video, maka semakin banyak poin yang diberikan.
- **(5 poin) [Deploy]** Aplikasi web yang telah dibuat perlu di deploy menggunakan Microsoft Azure Virtual Machine (VM) sehingga aplikasi dapat diakses secara publik melalui internet. Sebagai mahasiswa, terdapat kredit gratis sebesar \$100 dari Microsoft untuk dapat digunakan hanya dengan menggunakan email kampus ITB. Karena layanan ini menggunakan sistem pembayaran berbasis penggunaan (pay-as-you-go), penggunaan Virtual Machine akan mengurangi kredit yang dimiliki. Oleh karena itu, disarankan untuk mematikan (stop/deallocate) Virtual Machine ketika tidak digunakan agar kredit Azure tidak cepat habis.
- **(2 Poin) [Docker]** Aplikasi dijalankan menggunakan Docker baik untuk frontend maupun backend.

- **(6 Poin) [Animasi]** Menampilkan animasi penelusuran pohon DOM untuk segala pencarian.
- **(3 Poin) [Multithreading]** Mengimplementasikan multithreading pada algoritma BFS dan/atau DFS untuk melakukan traversal pohon DOM secara paralel. Proses traversal dibagi ke beberapa thread/worker sehingga beberapa node dapat diproses secara bersamaan.
- **(3 Poin) [LCA Binary Lifting]** Mengimplementasikan algoritma Lowest Common Ancestor (LCA) dengan teknik Binary Lifting pada pohon DOM yang dihasilkan dari proses parsing HTML. LCA digunakan untuk menentukan node leluhur terendah yang sama dari dua node dalam sebuah pohon. Dalam konteks DOM, fitur ini dapat digunakan untuk mengetahui elemen HTML terdekat yang menjadi parent bersama dari dua elemen tertentu. Penjelasan lebih lanjut dapat dilihat pada link berikut.



Bab 2

Dasar Teori



Bab 3

Aplikasi BFS dan DFS



Bab 4

Implementasi dan Pengujian

Bab 5

Kesimpulan dan Saran

5.1 Kesimpulan

Sebagai penutup, DOMTracer merupakan sebuah program yang mengemulasi cara kerja CSS Selector dalam memilih elemen-elemen bersesuaian dengan pada sebuah Document Object Model (DOM). Dalam program ini, DOM direpresentasikan sebagai struktur pohon dan kemudian diproses dengan menggunakan prinsip traversal graf/pohon berbasis algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS). Ide besar ini kemudian diterjemahkan menjadi rancangan sistem yang modular melalui pemisahan komponen parser, selector, dan traversal agar alur pemrosesan HTML menjadi jelas dan terstruktur.

Pada sisi implementasi, kami berhasil membangun alur lengkap dari masukan HTML, proses parsing menjadi struktur node, pencarian elemen berdasarkan selector, hingga penyajian hasil melalui antarmuka CLI. Fitur-fitur pendukung seperti pilihan sumber input (URL, HTML manual, dan file), pembatasan jumlah hasil (Top-N), serta pencatatan traversal log menunjukkan bahwa solusi tidak hanya berjalan secara fungsional, tetapi juga memperhatikan aspek kegunaan dan observabilitas proses.

Selama pengerjaan, banyak ilmu yang diperoleh, terutama dalam menghubungkan konsep algoritma yang dipelajari di kelas dengan kebutuhan rekayasa perangkat lunak nyata. Kami belajar bahwa pemilihan strategi traversal memengaruhi urutan kunjungan dan performa praktis, bahwa desain struktur data sangat menentukan kemudahan pengembangan fitur lanjutan, serta bahwa pengujian berbasis kasus berperan penting untuk menjaga ketahanan program pada input yang beragam, termasuk kasus batas dan input tidak valid.

Refleksi utama dari tugas besar ini adalah pentingnya berpikir sistematis dari definisi masalah, perancangan solusi, implementasi bertahap, hingga validasi hasil. Pengalaman ini menjadi bekal berharga untuk proyek-proyek di masa depan, khususnya dalam menerapkan algoritma secara kontekstual, menulis kode yang mudah dirawat, dan membangun kebiasaan verifikasi yang disiplin sebelum perangkat lunak digunakan lebih luas.

5.2 Saran

Program ini masih dapat dikembangkan lebih lanjut agar semakin kuat dan mendekati kebutuhan aplikasi nyata. Beberapa pengembangan yang disarankan adalah sebagai berikut.

1. Menambah dukungan CSS selector yang lebih lengkap, seperti selector gabungan yang lebih kompleks, pseudo-class tertentu, dan validasi sintaks selector yang lebih informatif.
2. Mengoptimalkan performa parser dan traversal untuk dokumen HTML yang sangat besar, misalnya melalui pendekatan streaming parse, caching hasil seleksi tertentu, atau strategi pruning pada proses pencarian.
3. Memperluas pengujian otomatis, termasuk benchmark performa, property-based testing, dan pengujian terintegrasi pada pipeline CI.

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Bandung, 19 April 2026

Moh. Hafizh Irham Perdana

Aufa Rienaldifaza Ahmad

Muhammad Aufar Rizqi Kusuma

Bab 6

Lampiran

6.1 Tabel Checklist

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan		
2	Aplikasi berhasil dijalankan		
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil		
4	Aplikasi dapat melakukan scraping terhadap web pada input		
5	Aplikasi dapat menampilkan visualisasi pohon DOM		
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran		
7	Aplikasi dapat menandai jalur tempuh oleh algoritma		
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log		
9	[Bonus] Membuat video		
10	[Bonus] Deploy aplikasi		
11	[Bonus] Implementasi animasi pada penelusuran pohon		
12	[Bonus] Implementasi multithreading		
13	[Bonus] Implementasi LCA Binary Lifting		

6.2 Tautan GitHub

Link GitHub

Bibliografi

- [1] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Massachusetts Institute of Technology, 2009.
- [2] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. John Wiley & Sons, 2009.
- [3] G. Nemhauser, L. Wolsey, and M. Fisher, “An analysis of approximations for maximizing submodular set functions—I,” *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.
- [4] D. S. Hochbaum and A. Pathria, “Analysis of the greedy approach in problems of maximum k -coverage,” *Naval Research Logistics*, vol. 45, no. 6, pp. 615–627, 1998.
- [5] R. Munir, “Algoritma greedy,” Lecture Notes, Institut Teknologi Bandung, 2025, accessed: 12-Feb-2026.